Optimization of k-Nearest Neighbors Classification through Systematic Data Sanitization

Tuguinay, Nanelle
College of Computer Studies and Engineering
Jose Rizal University
Pasig City, Philippines
nanelle.tuguinay@my.jru.edu

***Abstract*—This paper tests the importance of the process of data cleansing in the field of machine learning. In building a data cleaning pipeline to remove inconsistencies, outliers, and scale imbalances in a consumer dataset, we aimed to maximize the performance of a k-Nearest Neighbors (k-NN) classifier. From the experiments, we show that data cleaning sets a fundamental baseline for accuracy, but that hyperparameter tuning of the k-value is extremely important for achieving peak performance. For k=3, the accuracy was improved from a baseline of 42.86% to an optimum of 72.41%.**

***Keywords*—k-NN, Data Cleaning, MinMaxScaler, Hyperparameter Tuning, Overfitting.**

I. Introduction

The main aim of this study is to build and develop a predictive model based on the k-Nearest Neighbors (k-NN) algorithm for assessing Purchase_Likelihood. k-NN being a distance-based algorithm, relies on the proximity of the data points for prediction, and so, the main concern of this study is to ensure the data is clean and properly sanitized. [1].

A. Dataset Overview

A manual inspection of the adjusted_noisy_knn_300.csv dataset (Phase 1) revealed extensive "noise" that necessitated a preparation pipeline. Specific irregularities included:

- Inconsistency: For the categorical variable Gender, there were non-standard entries (e.g., "m", "Male", "Fem").
- Outliers: Age had impossible values like -10 and 200. Outliers markedly affect distance measurements [2].
- Redundancy: Annual_Income and Membership_Score were 0.9997 correlated, providing virtually the same information.
- Scale Imbalance: Annual_Income (121,577) and Support_Calls (10) had enormous numbers, impacting distance measures [1].

II. Methodology

A. Preparation Pipeline

Based on the code developed in Google Colab, a modular pipeline was implemented to sanitize the dataset:

- Deduplication: For the purposes of preventing the artificial weighting of data points we remove rows with duplicate Customer_IDs.

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler

df = pd.read_csv('/adjusted_noisy_knn_300.csv')

df = df.drop_duplicates(subset=['Customer_ID'])
```

Fig. 1. (*Data Loading and Initial Cleaning Source Code*)

- Outlier Management: Impossible ages and income values were recorded as NaN.
- Median Imputation: Instead of using the average value, which would be biased by existing outliers, we filled missing values with the median. This protects the integrity of the dataset and is a more constructive strategy than using mean imputation. [2].

```
df.loc[(df['Age'] > 100) | (df['Age'] < 0), 'Age'] = np.nan
df.loc[df['Annual_Income'] < 0, 'Annual_Income'] = np.nan

df['Age'] = df['Age'].fillna(df['Age'].median())
df['Annual_Income'] = df['Annual_Income'].fillna(df['Annual_Income'].median())
df['Total_Spend'] = df['Total_Spend'].fillna(df['Total_Spend'].median())
```

Fig. 2. (*Noise Filtering & Median Imputation Source Code*)

- Feature Engineering: A Purchase_Volume attribute was created by classifying Items_Purchased into three levels: Low, Medium, and High.

```
region_tax_data = pd.DataFrame({
    'Store_Region': ['North', 'South', 'East', 'West'],
    'Tax_Rate': [0.05, 0.07, 0.06, 0.08]
})
df = pd.merge(df, region_tax_data, on='Store_Region', how='left')
```

Fig. 3. (*External Data Merging (Enrichment) Source Code*)

- Min-Max Scaling: Using the MinMaxScaler, all numerical features were constrained to a range of [0, 1] in order to reduce the impact of any features that have a large magnitude on the model [1].
- One-Hot Encoding: Categorical strings were transformed into binary vectors in order to enable the processing of mathematical distances.

```
scaler = MinMaxScaler()
numeric_cols = ['Age', 'Annual_Income', 'Total_Spend', 'Items_Purchased', 'Last_Logon_Days', 'Support_Calls', 'Tax
df[numeric_cols] = scaler.fit_transform(df[numeric_cols])

df = pd.get_dummies(df, columns=['Gender', 'Device_Type', 'Store_Region', 'Purchase_Volume'])
```

Fig. 4. (*Feature Scaling & One-Hot Encoding Source Code*)

B. Experimental Setup

- Baseline Run: A “naive” run employing the unprocessed data and removing the non-numeric attributes, as well as the null values, without any form of scaling.

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

# Load the adjusted dataset
df_raw = pd.read_csv('/adjusted_noisy_knn_300.csv')

# --- Baseline Preparation ---
# 1. Keep only numeric columns (this automatically drops string 'noise')
# 2. Drop rows with any remaining Nan values
df_naive = df_raw.select_dtypes(include=['number']).dropna()

# Separate Features (X) and Target (y)
X_naive = df_naive.drop(columns=['Purchase_Likelihood'])
y_naive = df_naive['Purchase_Likelihood']

# Split the data
X_train, X_test, y_train, y_test = train_test_split(X_naive, y_naive, test_size=0.2, random_state=42)

# Train k-NN (k=5)
knn_naive = KNeighborsClassifier(n_neighbors=15)
knn_naive.fit(X_train, y_train)

# Evaluate
y_pred = knn_naive.predict(X_test)
print(f"Baseline Accuracy: {accuracy_score(y_test, y_pred):.2%}")
print("Baseline Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

Fig. 5. (*Baseline Source Code*)

- Optimized Run: The running of the model with the fully prepared pipeline.

```python
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# 1. Load the prepared dataset
# Replace with 'cleaned_knn_data.csv' or your specific file name
df_prepared = pd.read_csv('cleaned_knn_data.csv') # Changed path from '/cleaned_knn_data.csv'

# --- SAFETY CHECK: HANDLE REMAINING Nans ---
# This fixes the "ValueError: Input X contains Nans"
if df_prepared.isnull().values.any():
    print(f"Detecting {df_prepared.isnull().sum().sum()} missing values...")

    # Option A: Fill with median (Recommended to keep data)
    df_prepared = df_prepared.fillna(df_prepared.median(numeric_only=True))

    # Option B: Or drop them if very few remain
    # df_prepared = df_prepared.dropna()
    print("Missing values handled.")

# 2. Separate Features (X) and Target (y)
# Ensure the target name matches your dataset
target_col = 'Purchase_Likelihood'
X = df_prepared.drop(columns=[target_col])
y = df_prepared[target_col]

# 3. Split the Data (80% Training, 20% Testing)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 4. Train the k-NN Model (Optimized Run with k=5)
knn = KNeighborsClassifier(n_neighbors=15)
knn.fit(X_train, y_train)

# 5. Evaluate the Model
y_pred = knn.predict(X_test)

print("\n--- Optimized k-NN Model Resuls ---")
print(f"Accuracy Socre: {accuracy_score(y_test, y_pred):.2%}")
print("\nConfusion Matrix")
print(confusion_matrix(y_test, y_pred))
print("\nClassification Report:")
```

Fig. 6. (*Optimized Source Code*)

- Evaluation: Both runs used an 80/20 train-test split (n=58). The optimized model was iterated for values of k from k=1 to k=20.

III. Results and Experimental Analysis

A. Data Quality Comparison

Before preparation, the dataset was unusable due to 70+ missing entries and extreme scale differences. After the preparation pipeline, the data achieved 100% completeness, categorical uniformity, and mathematical normalization.

B. Performance Comparison

The results clearly show that the cleaning pipeline was the primary driver for model improvement:

k Value	Performance Comparison		
	Accuracy	***Observation***	***Confusion Matrix***
5	42.86%	Failed: High noise interference.	[[3, 1], [7, 3]]
1	67.24%	Sub-optimal: Overfitting (High Variance).	[[21, 10], [9, 18]]
3	72.41%	OPTIMAL PEAK: Best balance.	[[21, 10], [6, 21]]
7, 10, 15	68.97%	Plateau: High Stability.	[[22, 9], [9, 18]]
13, 20	67.24%	Degrading: Oversmoothing neighborhood.	[[21, 10], [9, 18]]

Fig. 7. (*Performance Comparison Table of the experimentation results*)

C. K-Optimization

The sweep from k=1 to k=20 identified the optimal neighbor parameter:

- k=1 (67.24%): Sub-optimal due to Overfitting; the model was too sensitive to local noise [3].
- k=3 (72.41%): The Optimal Peak, offering the best balance of local detail and noise filtering.
- k=5, 7, 10 (68.97%): A stable plateau demonstrating good generalization.
- k=13, 20 (67.24%): Performance drop due to Underfitting; the neighborhood became too broad, diluting the signal [3].

IV. Conclusion

One of the main technical issues I had to deal with was the ValueError: Input X contains NaN during optimized run, which was caused by the merging of the regional tax data. I addressed this by creating a “Safety Check” final imputation cell right before the data split to ensure the classifier would receive a full matrix.

There is a clear relationship between the amount of data prep that happens and the amount of performance the model will have. Although the k-NN algorithm is a powerful classifier, it is sensitive to the quality of the dataset. The increase in accuracy from 42.86% to 72.41% shows that systematic dataset sanitization, in particular, due to scaling and outlier removal, is the most significant enabler for effective deployment of a machine learning model.

References

[1] S. Raschka and V. Mirjalili, Python Machine Learning, 3rd ed. Packt Publishing, 2019. (Citations in Sections I-B and III-A regarding feature scaling).

[2] J. Han, M. Kamber, and J. Pei, Data Mining: Concepts and Techniques, 3rd ed. Morgan Kaufmann, 2011. (Citations in Sections I-B and III-A regarding outlier management).

[3] G. James, D. Witten, T. Hastie, and R. Tibshirani, An Introduction to Statistical Learning, Springer, 2013. (Citations in Section IV-C regarding the Bias-Variance Trade-off).